Remaining Useful Life Prediction of Aircraft Engines

NASA C-MAPSS FD001 · Applied Machine Learning (AML) Course Project, Phase 1

Team
Aryan · Akhil · Abhayanth · Srijan · Ashrith

Code
github.com/Abhayanthk/aml-rul-fd001

SUMMARY

We predict the Remaining Useful Life (RUL) of aircraft engines from their sensor readings, using NASA's C-MAPSS FD001 data. After removing 8 constant columns we keep 14 useful sensors and build 57 features from 10-cycle rolling windows. We compare two linear models (OLS and Elastic Net) with two tree models (Random Forest and gradient boosting). Gradient boosting is our best model, with an average test error of **18.5 cycles**. Close to failure the tree models are accurate but too optimistic; far from failure every model struggles, because a 10-cycle window cannot see long-term wear. This is the gap Phase 2 will address with an LSTM.

1 Introduction

1.1 The problem

Aircraft engines wear out as they run. The **Remaining Useful Life (RUL)** of an engine is the number of cycles it has left before it fails. For example, if an engine will fail at cycle 200 and we are now at cycle 150, its RUL is 50. The model only sees the sensor readings up to cycle 150 and must predict the 50.

1.2 Why RUL matters

Knowing the RUL lets maintenance be planned before a failure happens. Servicing an engine too early wastes parts that still had life in them; servicing it too late risks a breakdown. A good RUL estimate lets the engine be fixed just in time.

Why RUL matters

Fix too early → good parts are wasted

Fix too late → risk a breakdown

Know the RUL → fix it just in time

1.3 Goal

- Predict RUL from sensor readings.
- Compare linear models with tree models.
- Report the error separately for early and late stages, and find the "gap" that our features cannot capture.

1.4 Approach

The work is a five-step pipeline. Each step builds on the one before it, and each team member owns one step.

1 Data	→	2 Features	→	3 Evaluation	→	4 Trees	→	5 Final
RUL label, drop flat columns		10-cycle mean, std, slope		Engine split, linear baseline		Random Forest, boosting		Test once, early vs late, the gap
Aryan		**Akhil**		**Abhayanth**		**Srijan**		**Ashrith**

2 Data and RUL label

Aryan

We use the FD001 subset of NASA C-MAPSS. It has **100 training engines** that run until they fail and **100 test engines** that are stopped some time before failure.

For every training row, the RUL label is the engine's **last cycle minus the current cycle**. We do not cap it.

14 useful sensors (from 21)

The sensors stay flat and noisy for most of an engine's life, then drift steadily in roughly the last 100 to 125 cycles before failure (Figure 4). Eight columns almost never change, so they carry no information about wear; we removed them and kept 14 useful sensors.

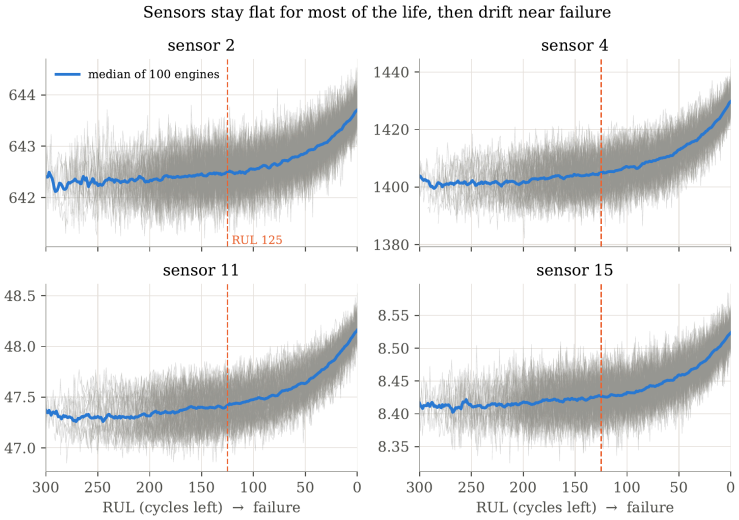


Figure 4 · four sensors vs RUL, 100 training engines: flat, then drifting near failure

3 Rolling features

Akhil

A single sensor reading is noisy, so for each sensor we summarise the **last 10 cycles of the same engine** with three numbers:

- **Mean** smooths out the noise.
- **Standard deviation** shows how much the sensor jumps around.
- **Slope** shows how fast the sensor is drifting, i.e. the speed of wear.

57 features: cycle + 14 sensors + 42 rolling

No leakage. The window only looks at past cycles and never crosses into another engine, so no future or foreign information leaks into a prediction.

Why 10 cycles? The window is long enough to remove the noise and short enough to work on every engine: even the shortest test engine has 31 cycles of history, so it always has a full window when we predict. Together with the cycle number and the 14 raw readings, this gives 57 features.

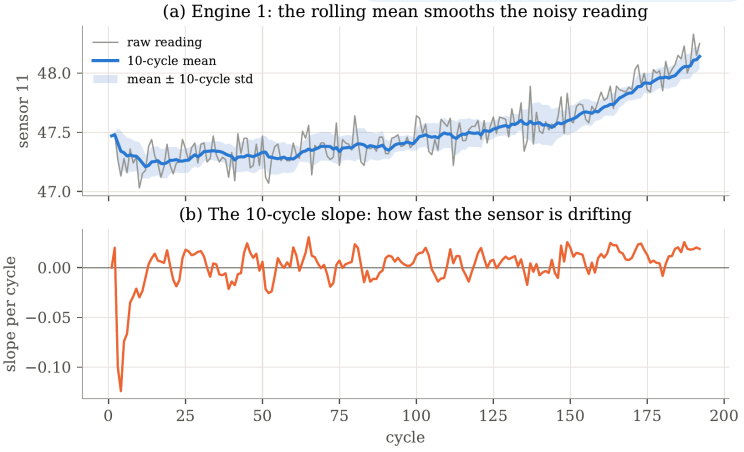


Figure 7 · sensor 11, engine 1: raw vs 10-cycle mean (top) and 10-cycle slope (bottom)

4 Linear baseline: OLS and Elastic Net

Abhayanth

Rows of the same engine are near-copies of each other, so we **split by engine, not by row**: the same engine never appears in both training and validation. Error is measured in cycles with two metrics: **MAE**, the average miss, and **RMSE**, which counts big misses more.

- **OLS** is the simplest baseline: a plain straight-line (least-squares) fit on the raw readings at the current cycle.
- **Elastic Net** is the same kind of fit on our engineered features, plus a penalty that keeps the weights small. Its **Lasso** part (L1) can set a weight to exactly zero and drop a feature; its **Ridge** part (L2) shrinks the weights.
- **Why we added it:** many sensors move together, which makes plain OLS unstable, and with 57 features we wanted the weak ones dropped. Lasso removed 7 of them.

Elastic Net beats OLS on both validation and test, so the features help even a linear model. But both models **underfit**: they are about 40 cycles off even on their own training data, because a straight line cannot follow the way engines wear.

Check	What it tells us	Better
MAE	the average miss, in cycles	**Elastic Net**
RMSE	like MAE, but big misses count more	**Elastic Net**
Training error	high for both: a straight line underfits	Neither

5 Tree models

Srijan

The sensors stay flat for most of an engine's life, then change fast near the end. A straight line cannot follow this "flat, then steep" shape, but trees can: they split the data into regions such as "still healthy" and "close to failure" and give each region its own prediction.

- **Random Forest** averages many trees built separately, which makes it more stable.
- **Gradient boosting** adds small trees one after another, each fixing the earlier mistakes, which makes it more accurate.

Both beat the linear models. We choose between them on **validation, not test**. On validation they are nearly tied, so we use two tie-breakers: near failure, boosting is too optimistic less often (69% vs 73%), and its error jumps less from training to validation, so it memorises less. The margins are small, so we treat them as tie-breakers. The test, used only once, agrees with the choice.

Check	Random Forest	Gradient boosting
Validation · pick the model here		
MAE	**25.0**	26.1
RMSE	40.2	40.2
Too optimistic near failure	73%	**69%**
Error jump, train → val	21 → 43	**30 → 43**
Test · report once		
MAE	19.0	**18.5**
RMSE	25.9	**25.3**

6 Final evaluation and the gap

Ashrith

Best model. Gradient boosting has the lowest error on the test set and, close to failure, is the least over-optimistic of the four models. We split points into **late** (125 or fewer cycles left) and **early** (more than 125 cycles left) to see where the models succeed and where they fail.

18.5 cycles: average test error (gradient boosting)

Late (near failure). Here the tree models are about 18 cycles off, so they are accurate where maintenance decisions are made. But most of their guesses still say there is a bit more life left than there really is, so repairs could come too late.

Early (far from failure) — the gap. Every model is about 50 cycles off, and the guesses flatten out. Our features only look at the last 10 cycles, so they cannot see how long the engine has been wearing out. Better models did not fix this, so the fix has to be in what the model sees.

Check	Late (≤ 125 cycles left)	Early (> 125 cycles left)
How far off	~18 cycles (trees)	~50 cycles (every model)
Main problem	Too optimistic	Guesses flatten out
Verdict	Good, but risky	**The gap → Phase 2**

The fix · Phase 2 → an LSTM that reads a much longer stretch of the raw sensor history, not just the last 10 cycles.

7 Team contributions

Member	What they built	Code
Aryan	Data loading, RUL label, EDA	load_data.py, notebook 01
Akhil	Rolling features	features.py, notebook 02
Abhayanth	Engine split, metrics, linear models	evaluation.py, notebook 03
Srijan	Random Forest and gradient boosting	notebook 04
Ashrith	Final evaluation, early/late, the gap	notebook 05

Code: github.com/Abhayanthk/aml-rul-fd001